Mobile Application Development III

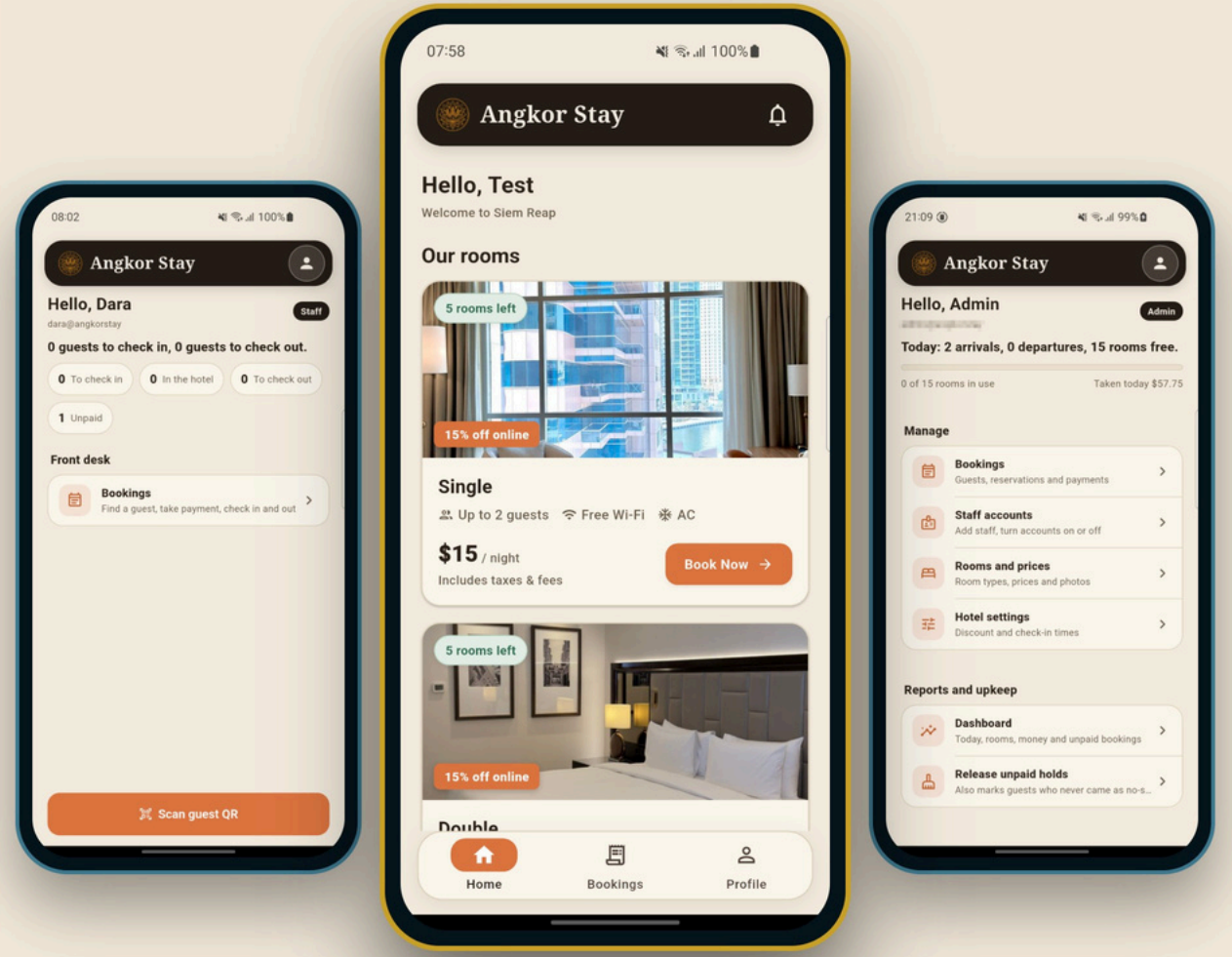
Angkor Stay

Hotel booking system

Lecturer

Oum SaoKosal

Final Group Project



Staff
Front desk

Guest
Book rooms

Admin
Manage hotel



Oum SaoKosal

Group members



Sa Rosmy



Meng Theth



Sokhorn Samnang



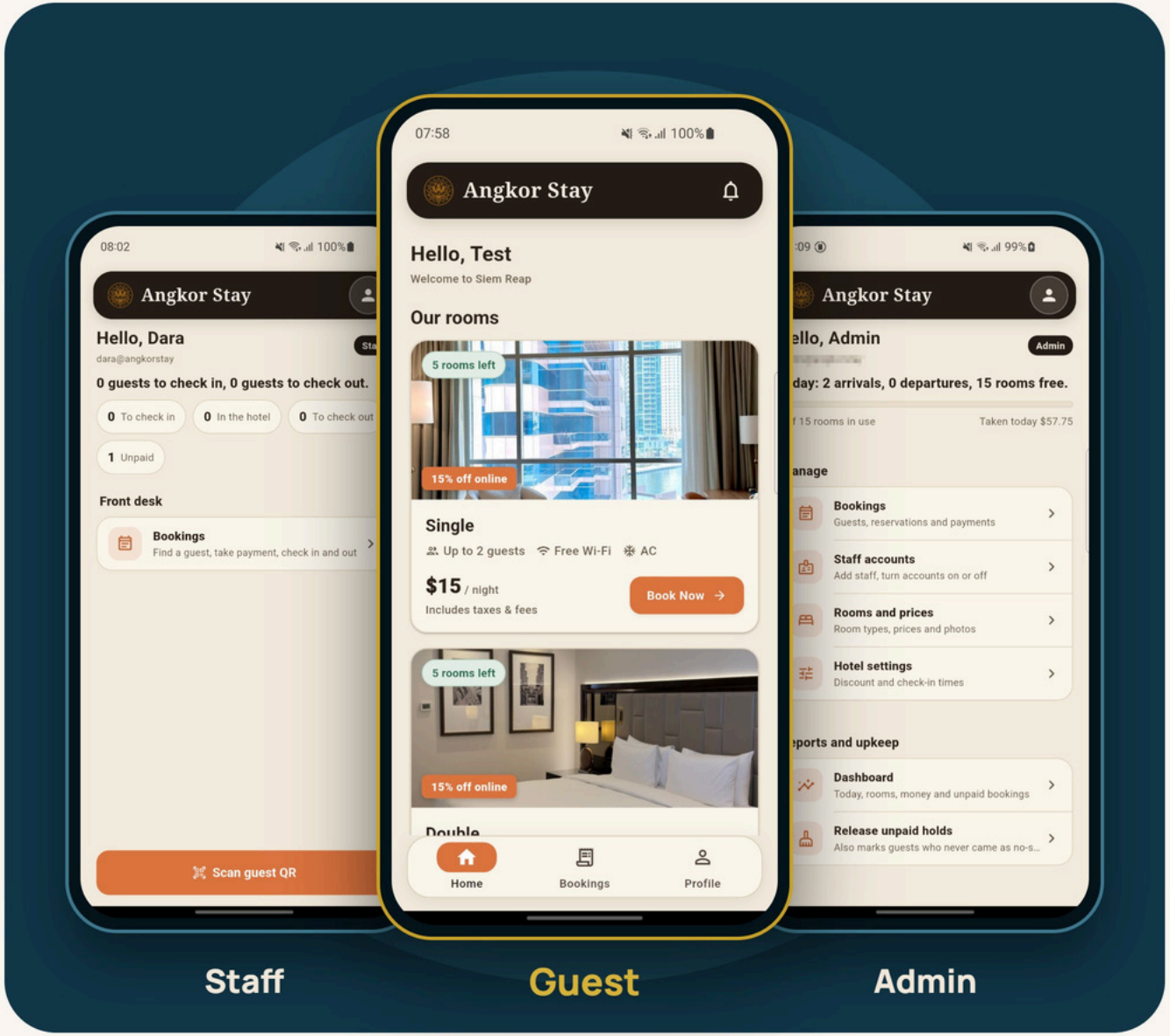
Heng Ratana

What is Angkor Stay?

A hotel booking system: one mobile app and one API (the server program the app talks to), used by three kinds of people.

Guest	Browses rooms, books and pays
Staff	Checks guests in and out, takes payment
Admin	Sets rooms, prices, staff and rules

15 rooms of **3** types, **10** database tables, a room hold of **5 minutes**, **15%** off when paying online.



One booking, from start to end

1	Guest	**Picks a room**	Chooses dates and guests. The server checks that a room is free.
2	Server	**Holds the room**	The booking is pending (waiting for payment). The room is kept for 5 minutes.
3	Guest	**Pays**	Card, QR code (15% off) or at the hotel. Online money is saved as a payment.
4	Staff	**Checks in**	On the arrival day staff scan the guest QR and check the guest in.
5	Staff	**Checks out**	The stay is finished. If the guest never came: no-show at 12:00 the next day.

How it is built

Flutter app

One app for three roles

Riverpod: shares data between screens

dio: sends requests to the API

Light and dark mode

JSON over HTTPS

Laravel 13 API

All the rules live here

Sanctum: login tokens

Services: the rules, in code

Jobs: free old holds, mark no-shows

SQL and row locks

PostgreSQL 18

The one source of truth

10 tables

Row lock: only one guest wins a room

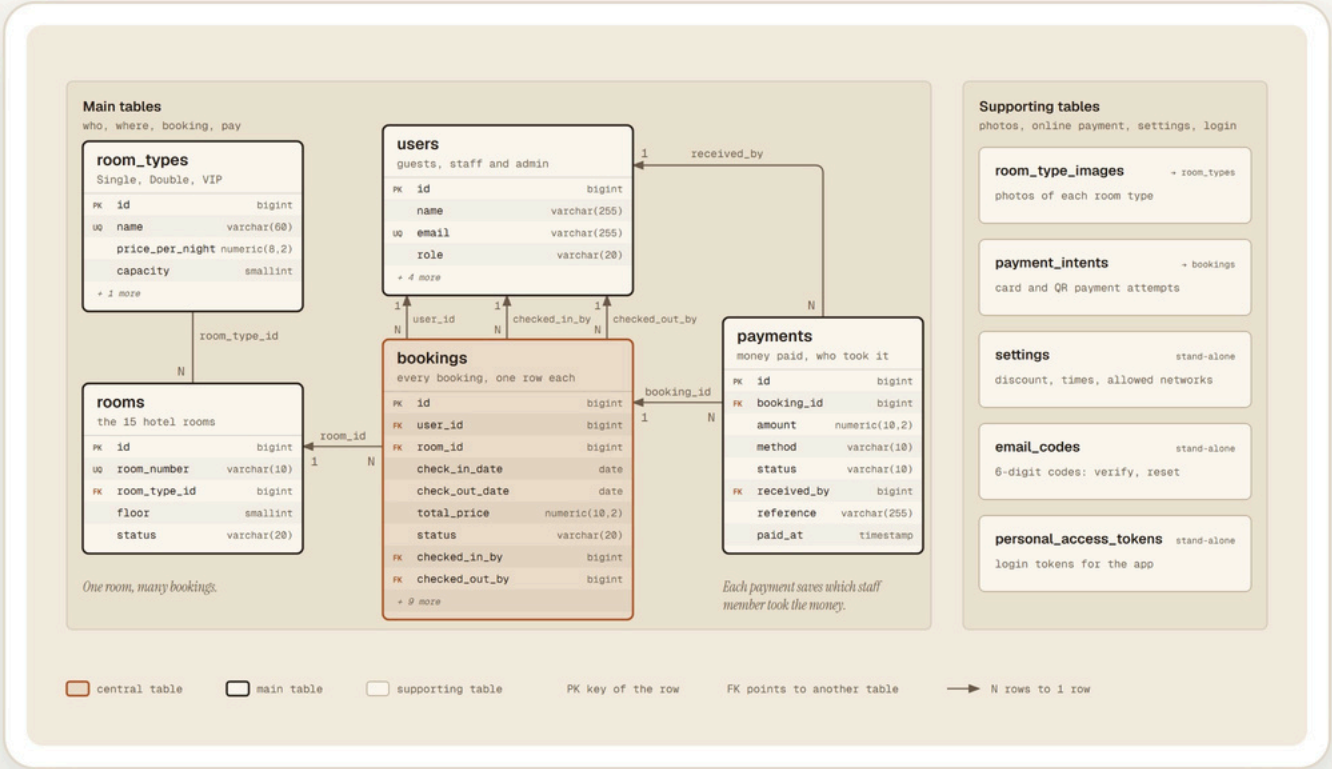
Links between tables protect history

Postman: 80 saved requests with tests

Gmail SMTP: 6-digit codes by email

Payments: card and KHQR (Cambodia bank QR), simulated

Database diagram: 5 main tables



5 main tables

users, room_types, rooms, bookings, payments. bookings is the centre.

5 supporting tables

room_type_images, payment_intents, settings, email_codes, personal_access_tokens.

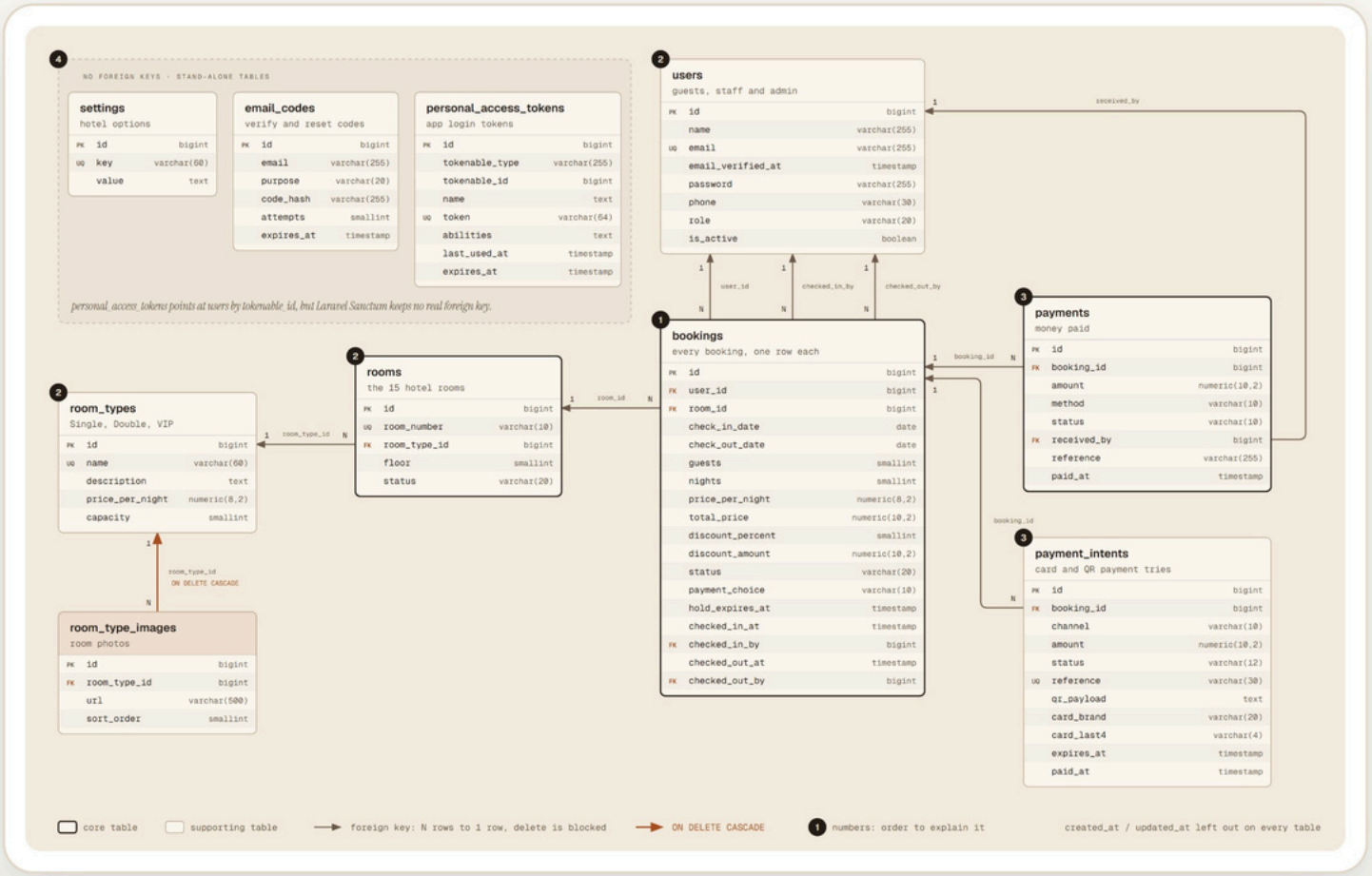
Price is copied

Each booking keeps the price from the day it was made, so a new price never changes old bookings. Each payment saves which staff member took it.

History is safe

A record cannot be deleted while another record still uses it, so nothing is lost by accident.

Full database: all 10 tables



Every field

Each field shows its type. Lines between tables show which record belongs to which.

Follow the numbers

Numbers 1 to 4 on the picture give the order to explain it.

Only photos auto-delete

Only room photos are deleted together with their room type. All other records are protected.

Secrets are hashed

Passwords and codes are saved scrambled (hashed). Nobody can read them.

Rules the system enforces, part 1

1

Check the room first

No payment page opens until a room is really free.

409 no_room_left

409 means conflict: the server says no before any payment page. Proof: Postman, slide 14.

2

5-minute hold

The room is kept for 5 minutes while the guest pays.

0:00 booked — 5:00 released

After 5 minutes the room is free again. Code: slide 10.

3

First guest wins

The database lets one guest through at a time, so two guests cannot get the same room.

60 guests → 5 rooms → 5 won

Stress test: 60 guests try at once. Only 5 rooms, so only 5 win.

Rules the system enforces, part 2

4

15% off online

The admin sets the discount. Paying at the hotel has no discount.



5

No-show at 12:00

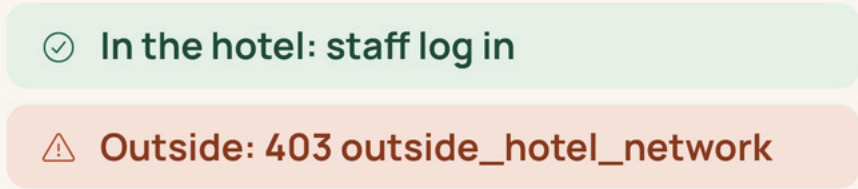
If the guest never comes, the room is freed at noon the next day and online money is refunded.



6

Staff on hotel Wi-Fi

Staff and admin can only log in from the hotel network, set by the admin.



Code: first guest wins

BookingController.php · store()

```php
$booking = DB::transaction(function () use ($request) {
    // Lock this room type: a second guest waits here
    $type = RoomType::whereKey($request->room_type_id)
        ->lockForUpdate()->firstOrFail();                    // 1

    $room = $this->availability->freeRooms($type, $in, $out)
        ->orderBy('room_number')->first();                   // 2
    if (! $room) {
        return response()->json(['code' => 'no_room_left'], 409);   // 3
    }
    return Booking::create([ /* ... */
        'status' => 'pending',
        'hold_expires_at' => now()->addMinutes(5),           // 4
    ]);
});
```

1. **Lock the room type**
   Two guests at the same moment cannot both win.

2. **Server picks the room**
   First free room by room number.

3. **No room left: 409**
   Checked before any payment page opens.

4. **5-minute hold**
   The booking starts as pending and the hold ends in 5 minutes.

Code: cancel and refund

BookingController.php · cancel()

```php
DB::transaction(function () use ($request, $id) {
    $booking = Booking::where('user_id', $request->user()->id)
        ->lockForUpdate()->findOrFail($id);                          // 1

    if ($booking->status === Booking::STATUS_CONFIRMED) {
        // sends the money back first; throws if refused
        $this->online->refundOnlinePayments($booking);               // 2
    }
    // closes a payment page that is still open
    $this->online->abandon($booking);                                // 3

    $booking->update(['status' => 'cancelled',                       // 4
                      'hold_expires_at' => null]);
});
```

1 Only my booking
Found by the guest's own id, then locked.

2 Refund first
The money goes back before anything is cancelled.

3 Close open payment
A payment page still open is closed.

4 Cancel last
A failed refund throws, so nothing is cancelled.

Code: the Flutter app calls the API

booking_repository.dart · create()

```dart
Future<Booking> create({                                  // 1
  required int roomTypeId,
  required Stay stay,
}) => _call(() async {                                    // 2
  final res = await dio.post('/bookings', data: {         // 3
    'room_type_id': roomTypeId,
    'check_in_date': stay.checkInText,
    'check_out_date': stay.checkOutText,
    'guests': stay.guests,
  });
  return _booking(res);                                   // 4
});
```

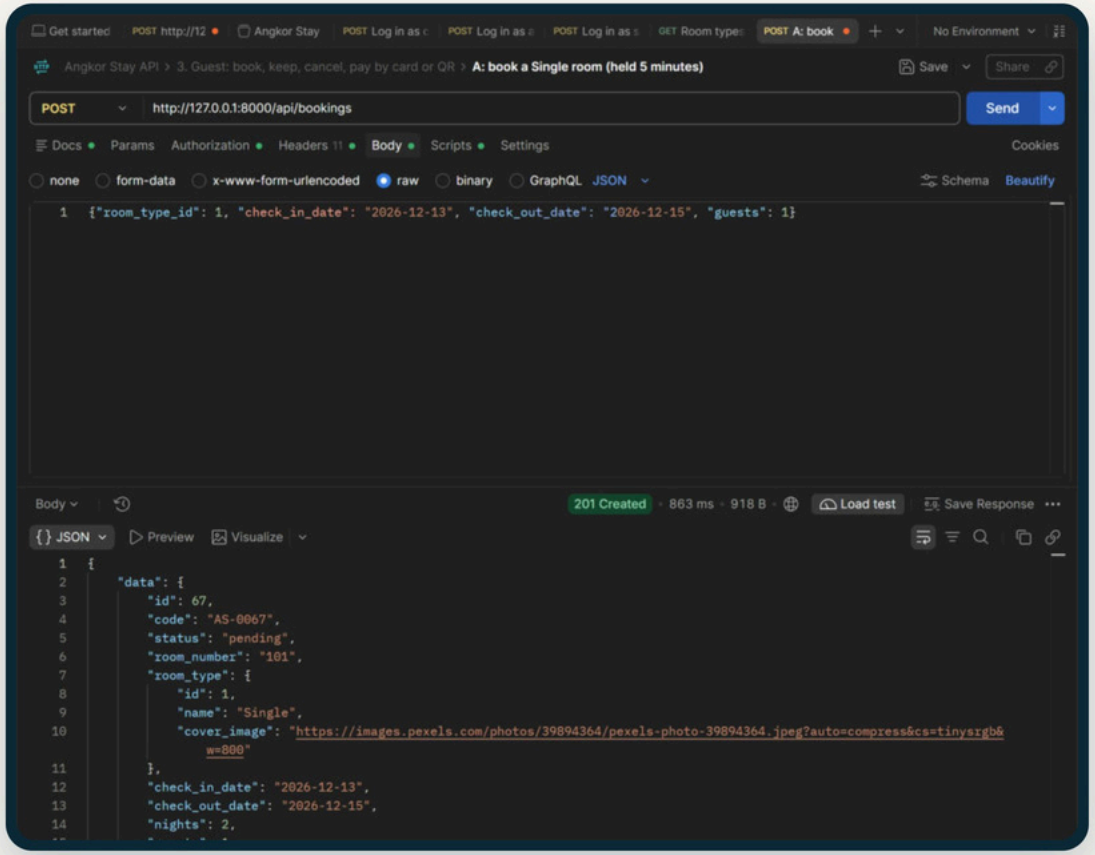
1. **One repository**
   Riverpod gives every screen the same one.

2. **One error type**
   Errors become an ApiError with the server's message.

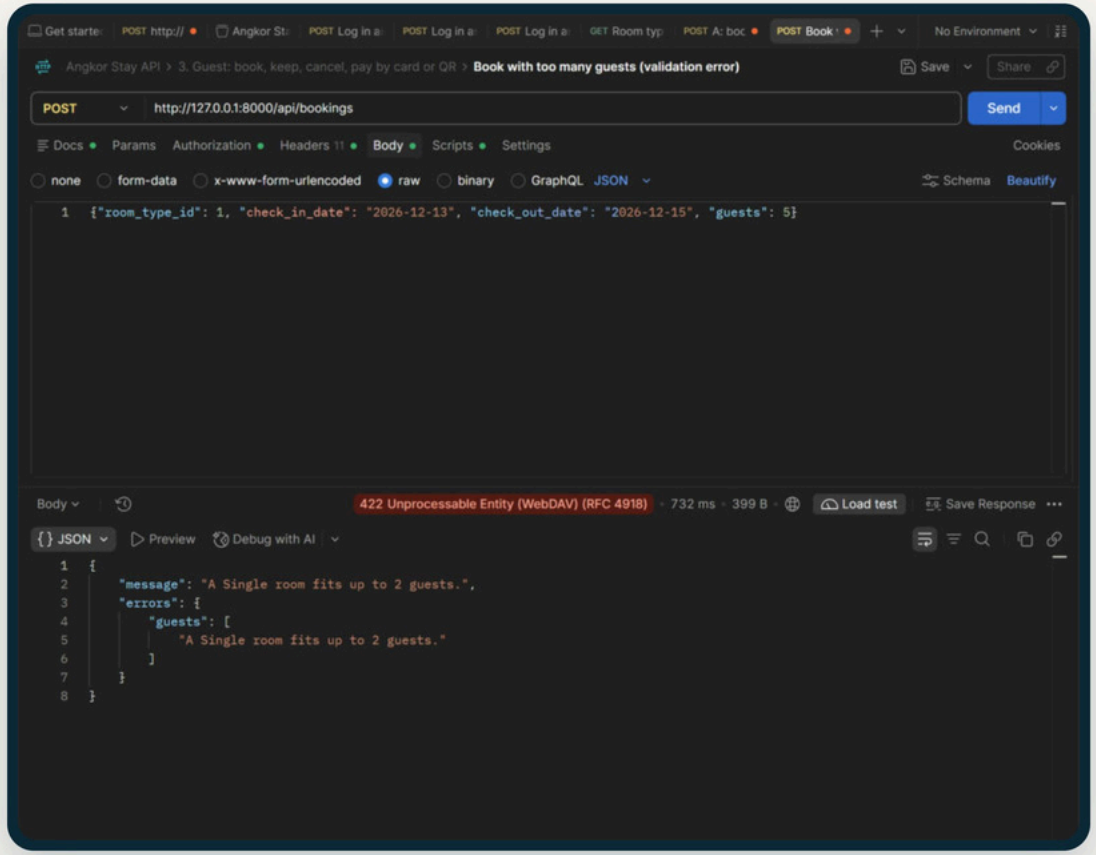
3. **dio sends the token**
   POST /bookings with the login token.

4. **JSON becomes a Booking**
   The screen gets a ready object.

Postman: the guest books a room

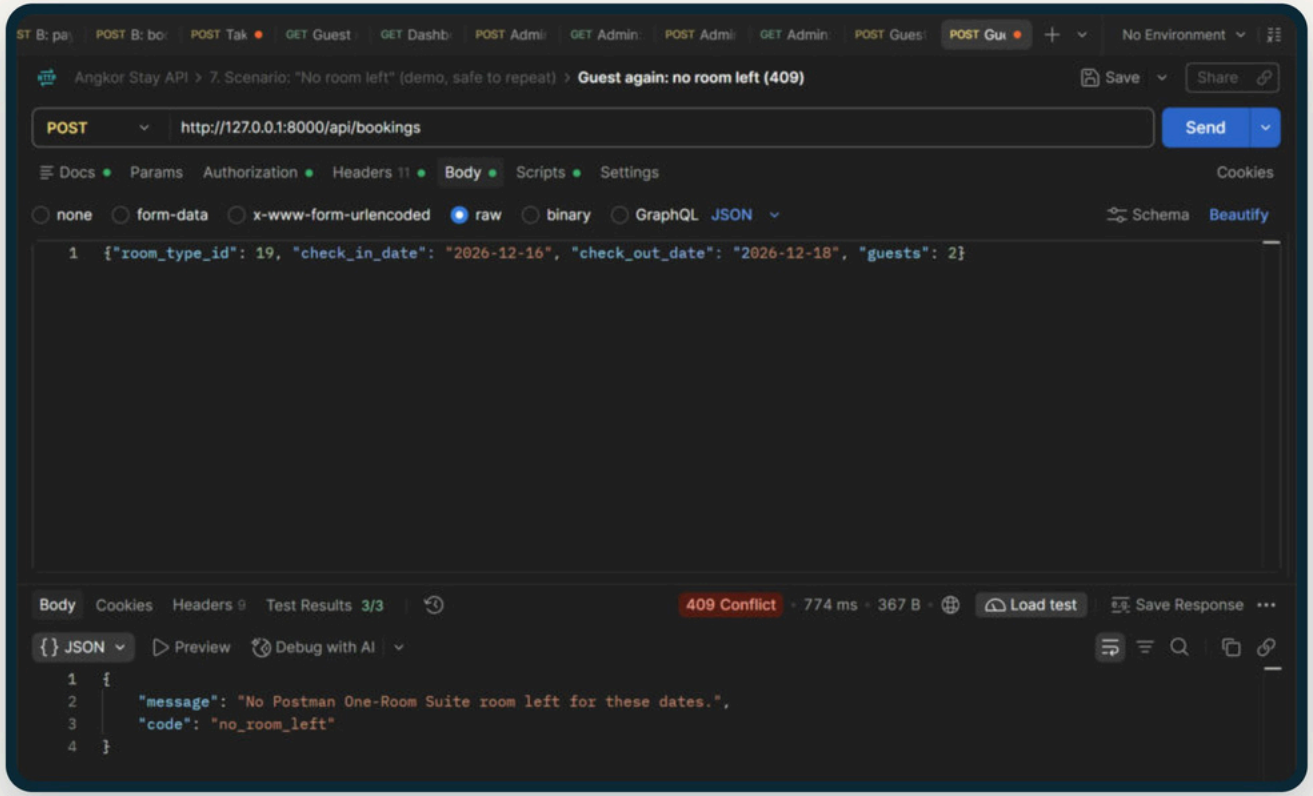


201 Created: room booked, status pending, 5-minute hold



422 Too many guests: “A Single room fits up to 2 guests.”

Postman: the guest finds no room left



409 no_room_left: checked before any payment. Test results 3/3.

A guest asks for a room type that has no free room on those dates.

The server answers 409 with the code no_room_left.

No payment page opens. The lock from the code slide keeps it fair. ("Postman One-Room Suite" is a test room type with only one room, made for this demo.)

Book a room in a few taps

Browse rooms

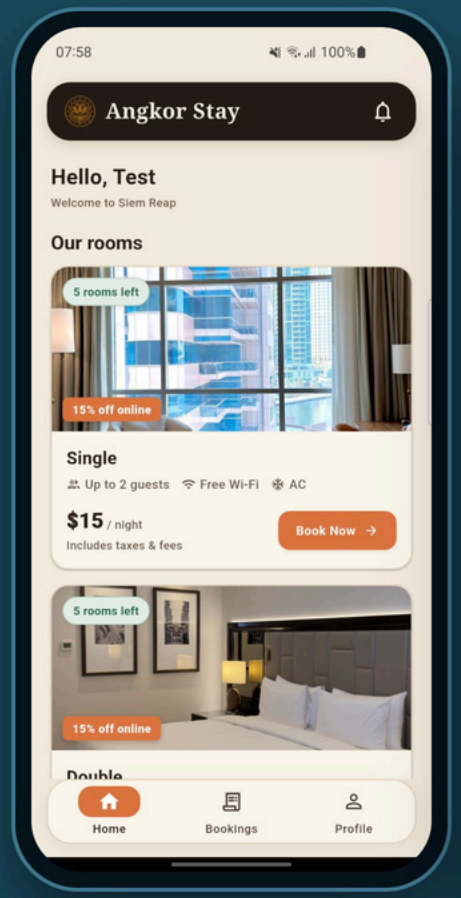
See photos, price and free rooms.

Pick your dates

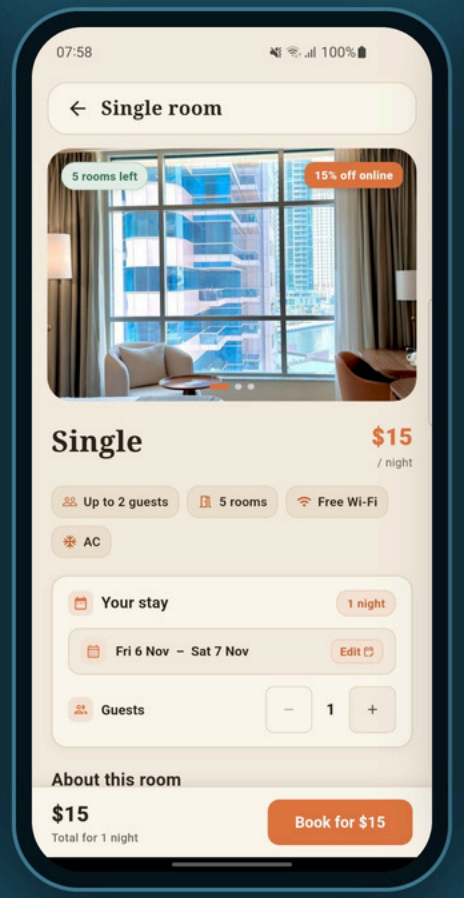
Choose dates and guests. The total price shows.

Hold the room

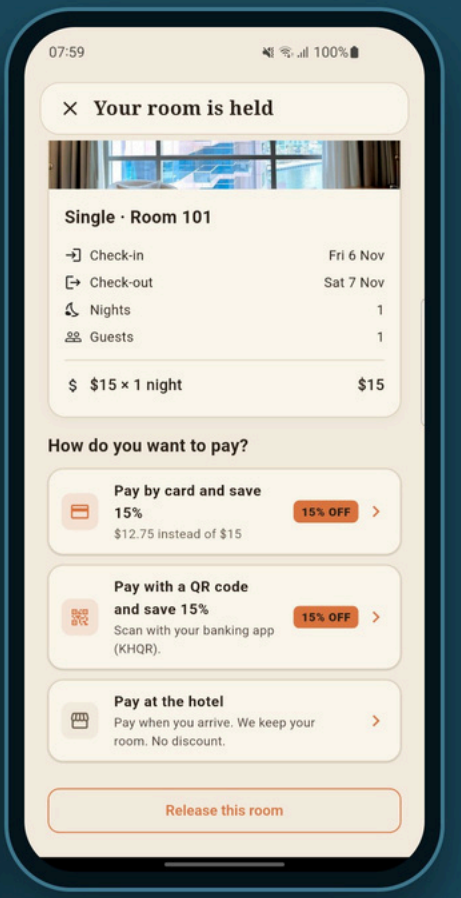
The room is kept for 5 minutes while you pay.



Rooms and photos



Room detail



5-minute hold

Track every booking

Check the status

Waiting for payment, confirmed or finished.

Show the QR receipt

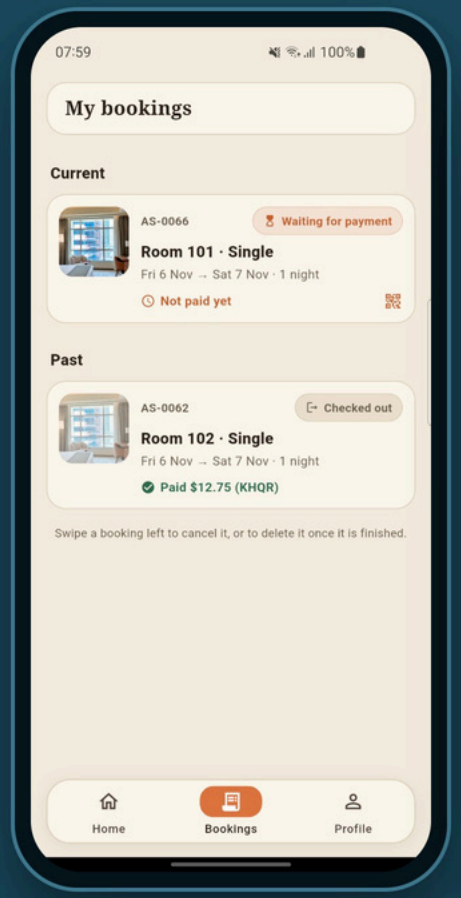
Staff scan it at the front desk.

Cancel before check-in

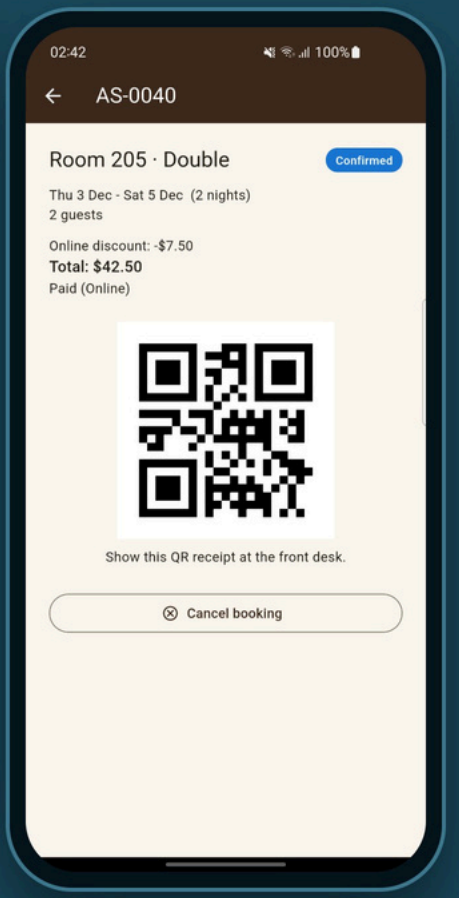
Online payments are refunded first.

Edit your profile

Change your name, phone or password.



My bookings



QR receipt

Run the front desk

Find a booking

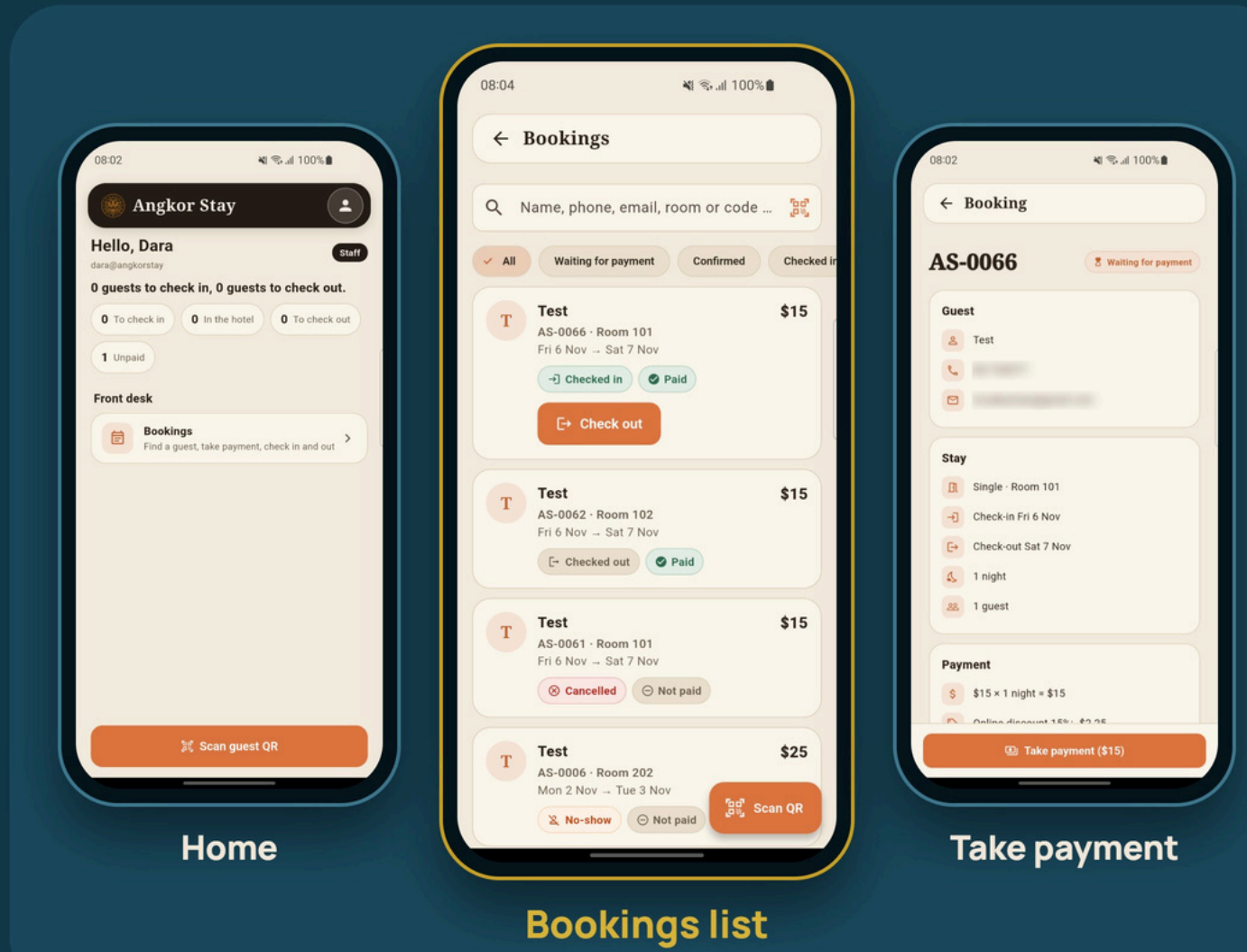
Search by guest name or booking code.

Take payment

Cash, card or KHQR. The amount and staff name are saved.

Check in and out

Scan the guest's QR code, then one tap.



See the whole hotel

See the dashboard

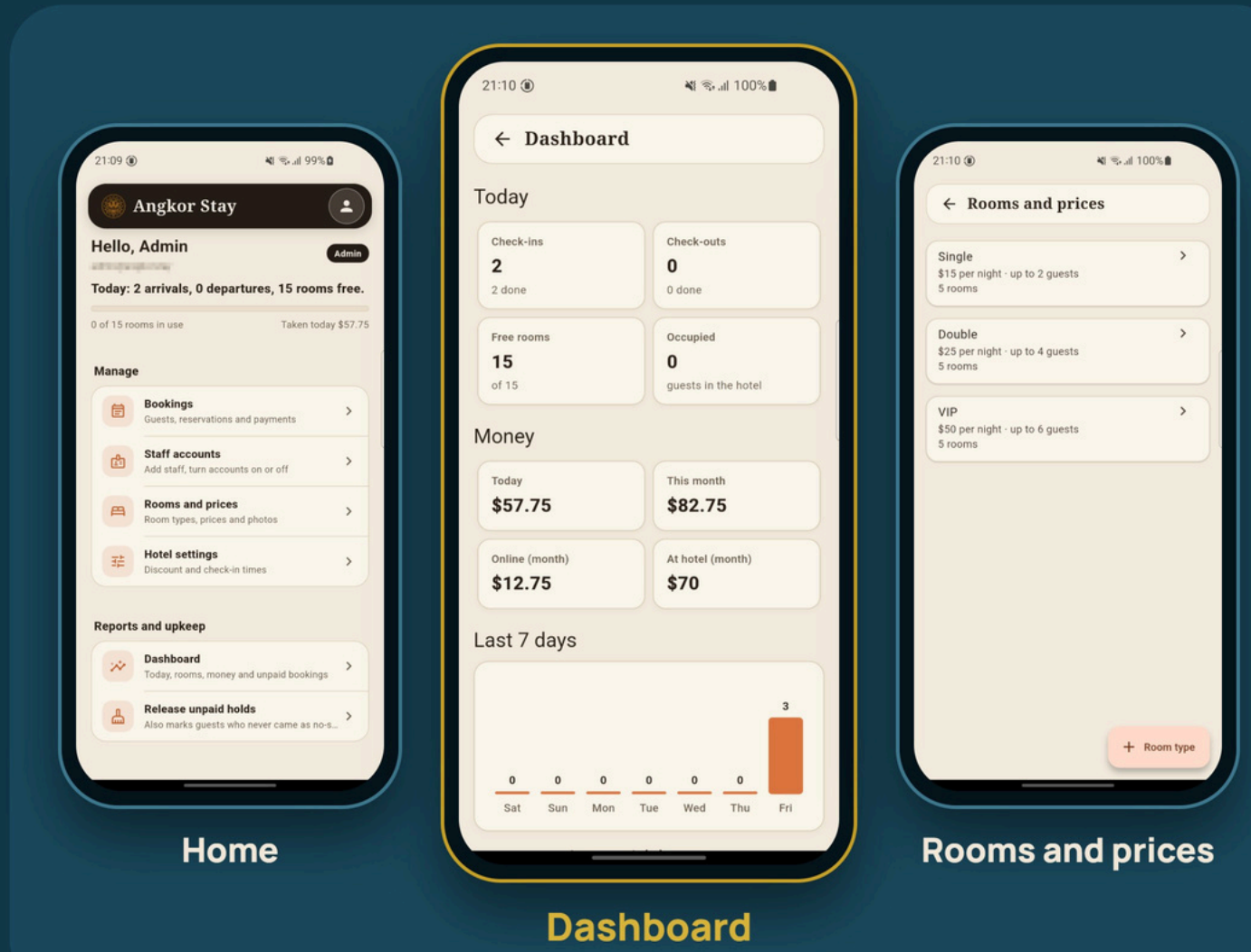
Free rooms, arrivals and money.

Set rooms and prices

Add or change room types and photos.

Manage staff and rules

Create staff accounts. Set the discount, times and network.



Testing: how we know it works

293	**Backend tests** Checks the Laravel API on a PostgreSQL test database
222	**App tests** Checks the Flutter screens and their logic
113	**Postman checks** 80 requests saved in Postman. 70 run automatically with Newman (Postman command-line tool)
1	**Winner when two guests book the same room** Both tried at the same moment: only one gets it

Also checked by hand on a real Android phone against the running API.

Source code and video

Source code

github.com/HengRatana404/angkor-stay

Presentation video

[paste the video link here]

Every member shows their face and says their name.

Thank you

Questions?



Angkor Stay

Flutter app, Laravel API, PostgreSQL